

PCA of RNA-seq Samples

2026-04-17

Introduction

Sample-level PCA on transformed RNA-seq counts is the standard quality-control step before differential-expression analysis. It reveals whether biological samples cluster by experimental condition (the desired outcome), by batch or technical covariates (a confounding signal that needs correction), or by neither (which can suggest excessive noise or sample mix-ups). Running PCA early — before any modelling — prevents the most consequential downstream errors and motivates the design choices for the differential-expression step.

Prerequisites

A working understanding of principal components analysis, RNA-seq count data, and the role of variance-stabilising transformations in equalising the variance-mean relationship of count data.

Theory

Raw RNA-seq counts have variance that grows with the mean, violating the assumption of equal variance that PCA implicitly relies on. Variance-stabilising transformations (VST and the older `rlog` in DESeq2) apply a non-linear transformation that approximately equalises variance across the expression range; the result is suitable for PCA, clustering, and other downstream methods that assume homoscedastic input.

PCA on the top-variance genes (typically the top 500 or so) highlights the dominant sources of variation. The proportion of variance explained by each PC indicates how much of the total variability is captured, and the score plot — samples coloured by metadata variables — exposes which design factors drive that variability.

Assumptions

Sufficient samples (rule of thumb: at least 6, ideally more) for meaningful PCA; counts have been variance-stabilised; the chosen number of top-variance genes is large enough to capture the structure but small enough to suppress noise.

R Implementation

```
library(DESeq2); library(ggplot2)

set.seed(2026)
counts <- matrix(rpois(20000 * 12, lambda = 30), nrow = 20000)
colnames(counts) <- paste0("s", 1:12)
coldata <- data.frame(condition = rep(c("A", "B"), each = 6),
                      batch = rep(c("X", "Y"), 6))

dds <- DESeqDataSetFromMatrix(counts, colData = coldata, design = ~ condition)
vsd <- vst(dds, blind = TRUE)

plotPCA(vsd, intgroup = c("condition", "batch"))
```

Output & Results

`plotPCA()` returns a ggplot of samples projected onto PC1 and PC2 with axis labels showing the proportion of variance explained. Colouring by `condition` and shape by `batch` (or vice versa) makes both factors visible simultaneously.

Interpretation

A reporting sentence: “Sample-level PCA on VST-transformed counts (top 500 variance genes) showed PC1 capturing 40 % of variance and clearly separating treatment conditions, while PC2 (12 %) showed a weaker batch effect. We included `batch` in the differential-expression design to adjust for it.” Always report the proportion of variance explained and what each PC corresponds to substantively.

Practical Tips

- Use `blind = TRUE` for QC PCA so the variance-stabilisation does not shrink toward the design; for downstream DE, the design-aware (`blind = FALSE`) transformation is appropriate.
- If batch dominates PC1, include batch in the DESeq2 (or limma) design; if batch overlaps perfectly with condition, the experiment is confounded and cannot be salvaged.
- Plot multiple metadata colourings (sex, age, RIN, library prep date) on the same PCA scatter to reveal hidden structure; outlier samples often correspond to specific covariates.
- Complement PCA with a sample-sample distance heatmap (`pheatmap(as.matrix(dist(t(assay(vsd)))))`); the two views agree on the broad story but emphasise different details.
- Outliers on PCA often indicate QC failures (low library complexity, contamination); investigate and consider exclusion before differential analysis.
- Compare PCA on raw counts, log-counts, and VST; raw counts have variance dominated by highly expressed genes, log-counts overcorrect, VST is calibrated for downstream analysis.

R Packages Used

DESeq2 for `vst()`, `rlog()`, `plotPCA()` and the broader RNA-seq workflow; `pheatmap` for sample-distance heatmaps; `ggplot2` for custom PCA plots; `PCAtools` for richer PCA visualisations including biplots and loading plots.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- RNA-seq with DESeq2 / edgeR
- Enrichment analysis (GSEA / ORA)
- scRNA-seq with Seurat